

Daten im Hauptspeicher



Data + BSS:

- Enthalten globale und statische Programmvariablen, aufgeteilt in einen *read-only* und einen *read-write* Abschnitt
- Größe ändert sich zur Laufzeit nicht

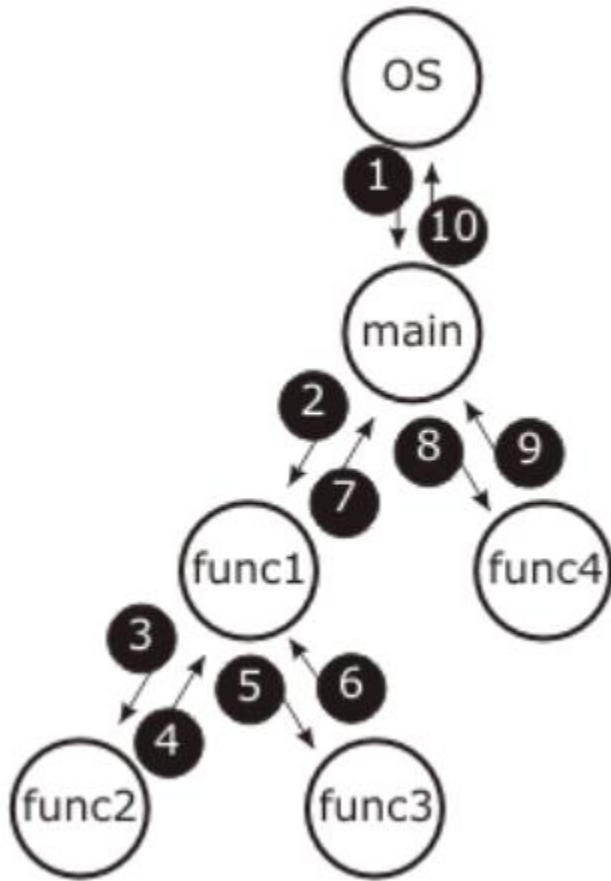
Stack:

- Enthält lokale und temporäre Daten von aufgerufenen Methoden
- Wächst und schrumpft mit der Anzahl der aufgerufenen Methoden

Heap:

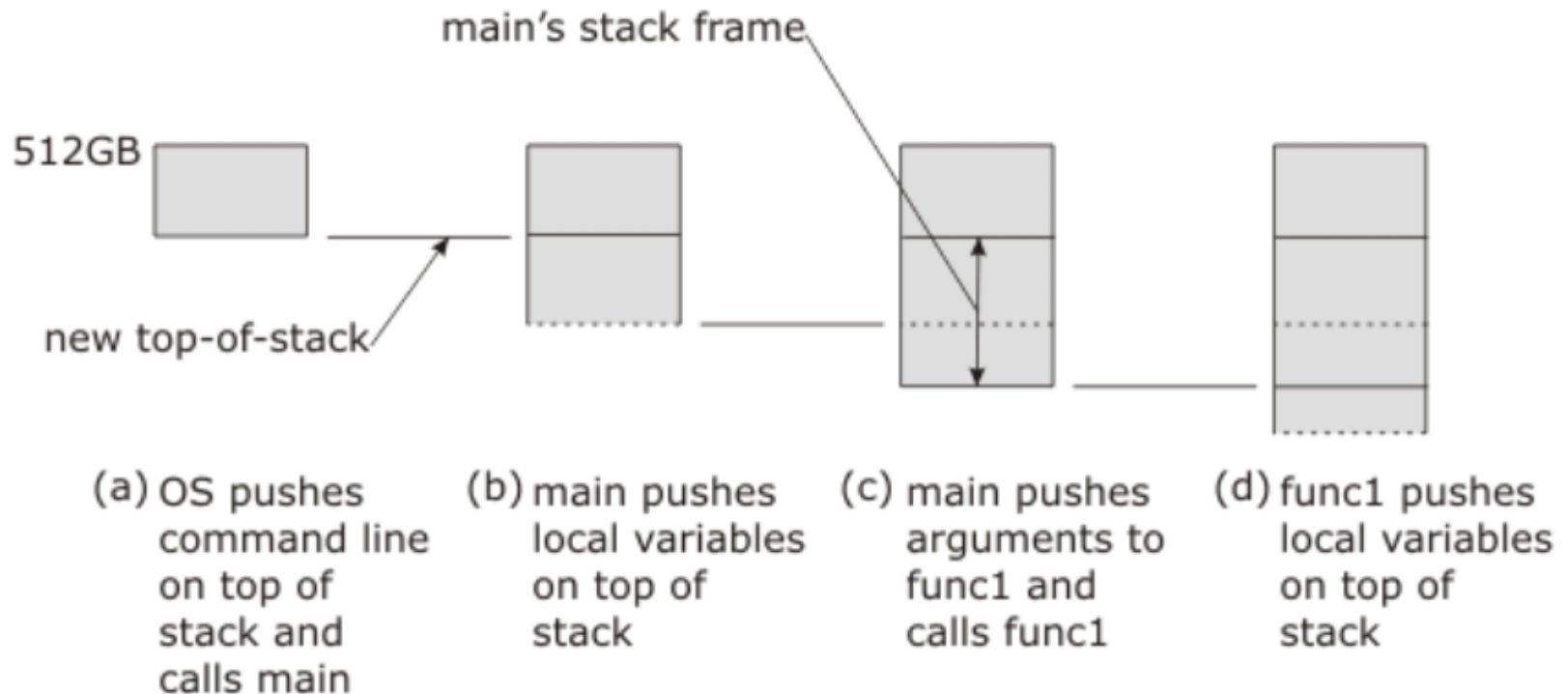
- Dynamischer Speicher, der für die Anwendungsprogrammierung zur Verfügung steht (Arrays, Bilddaten, Objektinstanzen, ...)
- Schlüsselwort „**new**“ (C/C++ auch „**delete**“)

Stack-Beispiel: Aufrufkette

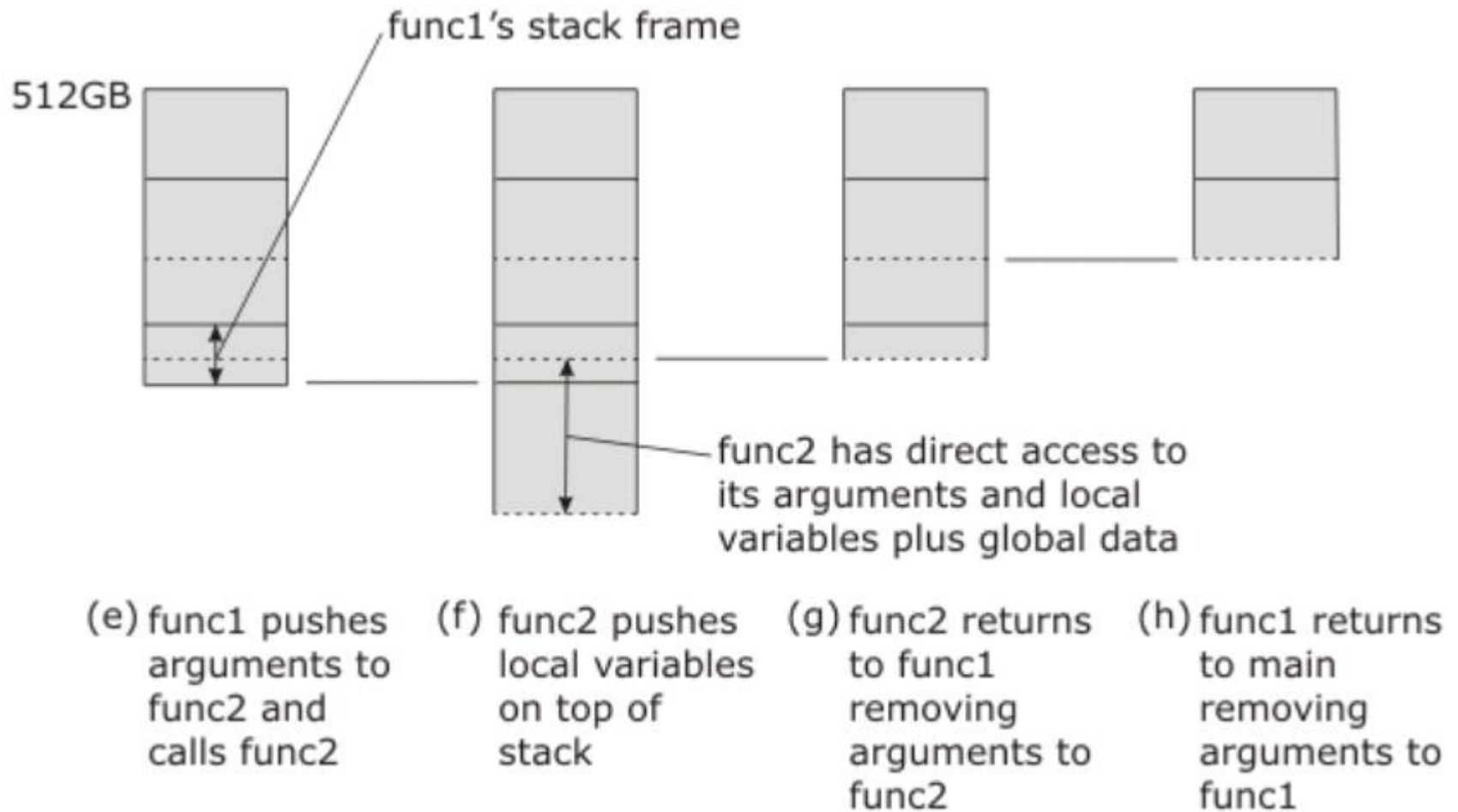


1. Das Betriebssystem startet die Main-Funktion eines Programms
2. Methode „func1“ wird aufgerufen
3. „func2“ wird aufgerufen
4. „func2“ wird beendet
5. „func3“ wird aufgerufen
6. „func3“ wird beendet
7. „func1“ wird beendet
8. „func4“ wird aufgerufen
9. „func4“ wird beendet
10. Main wird beendet

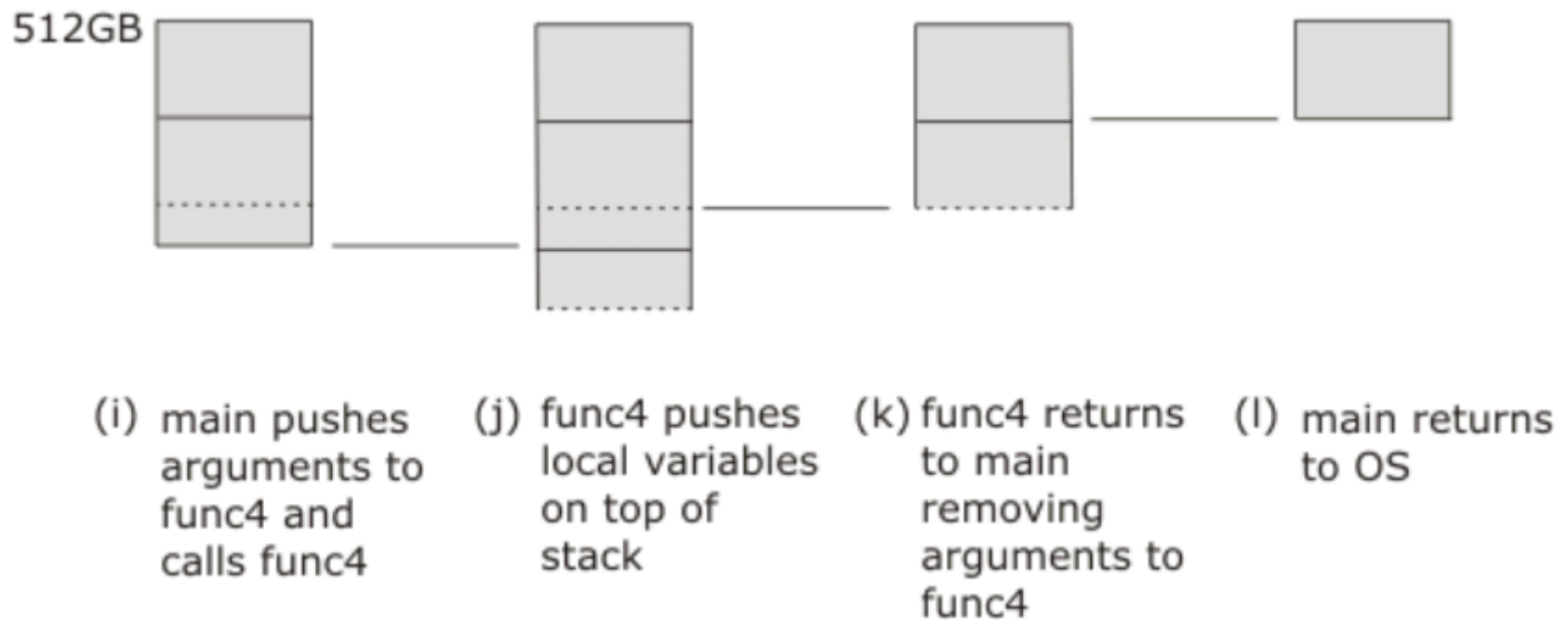
Methodenaufrufe: Stack Frames



Methodenaufruf und -rückkehr



Stack: Programmende



Heap: Dynamischer Speicher

Ein Programm fordert Heap-Speicher beliebiger Größe an (Dynamische Speichieranforderung):

- Speicher allokieren: **malloc**-Funktion (C#,C++, Java: „**new**“)
- Speicher freigeben: **free**-Funktion (C++: „**delete**“)

Automatische Speicherbereinigung

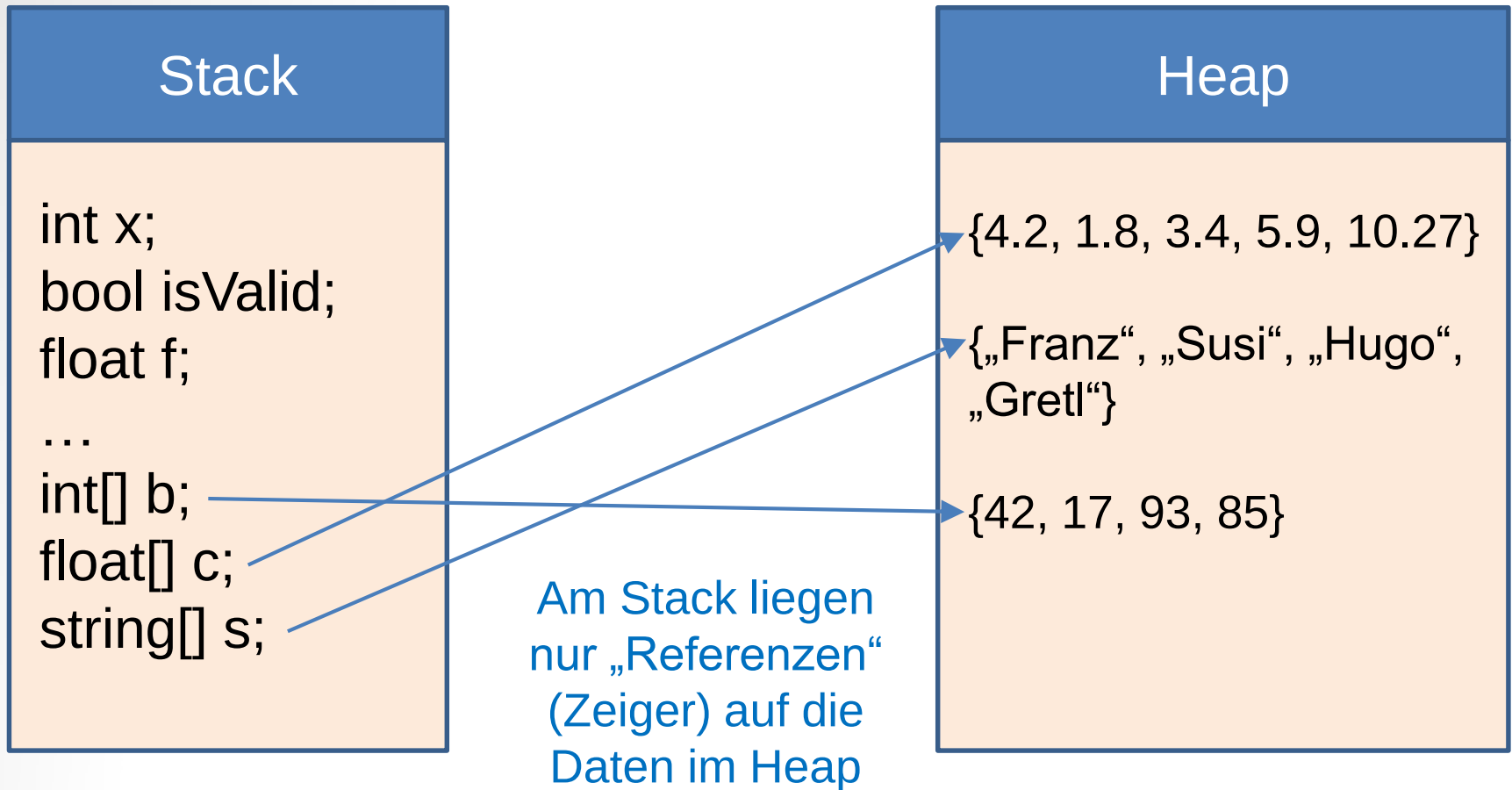
- Programmiersprachen wie C# und Java enthalten einen **Garbage Collector**, der dynamischen Speicher, der nicht mehr benötigt wird, regelmäßig wieder freigibt (=> ans Betriebssystem zurückgibt)

Manuelle Speicherverwaltung

- Das Anwendungsprogramm ist selbst verantwortlich, dass allozierter Speicher auch wieder freigegeben wird
- Fehlende Freigaben: Häufige Fehlerursache in Sprachen wie C/C++!

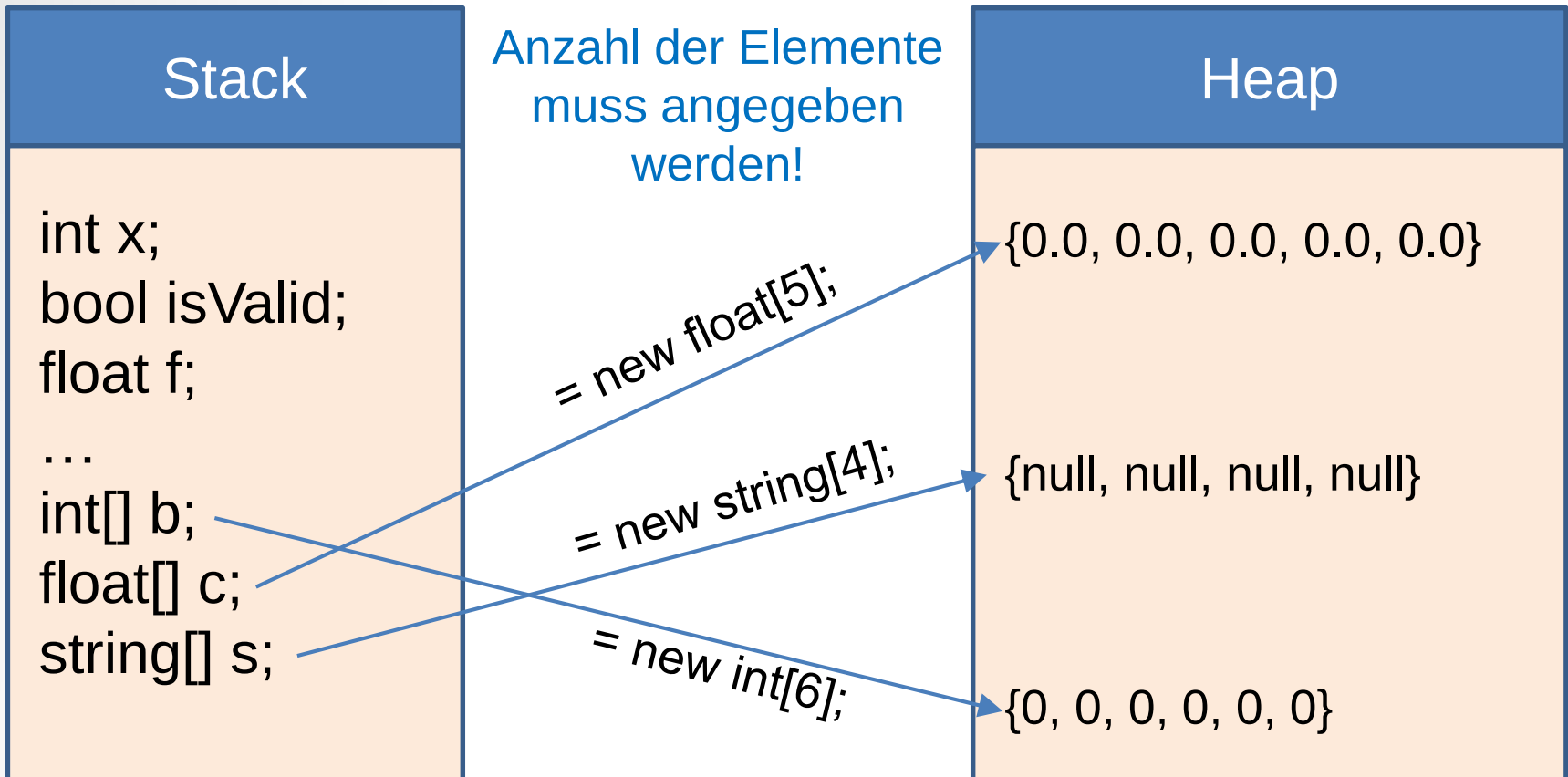
Stack oder Heap?

Große Mengen an Daten werden am Heap allokiert:



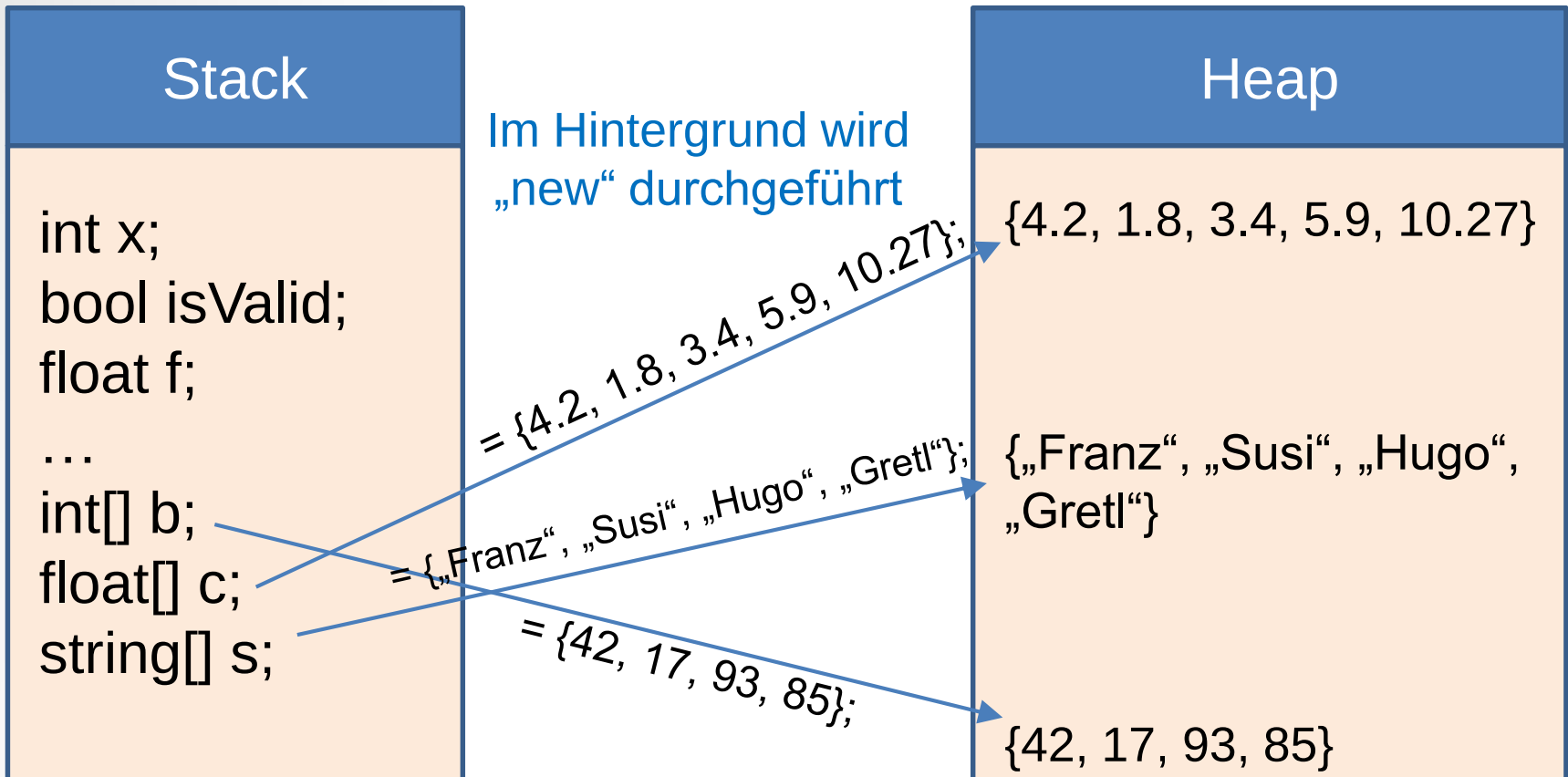
Heap: Speicherallokation

Einer Array-Referenz muss mit „**new**“ Heap-Speicher zugewiesen werden:



Heap: Allokation durch Initialisierung

Einer Array-Referenz kann auch bei der Initialisierung schon fertig befüllter Speicherbereich zugewiesen werden:



Werttyp - Referenztyp

```
int x;  
bool isValid;  
float f;  
char c;
```

Werttyp (*value type*)

- **Primitive Datentypen** (z.B. bool, char, int, double) sind Werttypen, weil die notwendige Speichergröße zur Übersetzungszeit bekannt ist und die Variable direkt den Wert enthält.
- Werttyp-Variablen liegen vollständig im Stack.

```
int[] b;  
float[] c;  
string[] s;  
string t;
```

Referenztyp (*reference type*)

- **Komplexe Datentypen** (z.B. Strings, Arrays, Objektinstanzen) sind Referenztypen, weil die Nutzdaten erst zur Laufzeit am Heap allokiert werden. Die Größe eines Arrays ist zur Übersetzungszeit nicht (immer) bekannt.
- Die Variable enthält nur eine Heap-Referenz.
- Referenztypen kann der Wert „**null**“ zugewiesen werden. (Hier können daher auch ***NullReferenceExceptions*** passieren)